

3-Day SQL Catch-Up Plan

Overall strategy

Use this progression:

Understand → Copy a pattern → Modify it → Solve independently → Explain it

Don't start with theory.

For every SQL topic, use the same cycle:

CONCEPT



ONE SIMPLE EXAMPLE



GUIDED QUERY



MODIFY THE QUERY



NEW PROBLEM



EXPLAIN WHY IT WORKS

The trainee should work primarily with **one consistent business schema**—for example:

customers

orders

order_items

products

employees

departments

This avoids wasting mental energy learning a new schema for every exercise.

DAY 1 — SELECT → FILTER → AGGREGATE → JOIN

Goal

By the end of Day 1, he should be comfortable writing ordinary queries and combining tables.

Session 1 — SQL foundation

30 minutes

Focus only on:

```
SELECT
FROM
WHERE
ORDER BY
DISTINCT
LIMIT
```

Then introduce expressions:

```
SELECT
    product_name,
    price,
    price * 1.18 AS price_with_tax
FROM products;
```

Practice ladder

Start extremely easy:

1. Display all customers.
2. Display selected columns.
3. Find customers from a particular region.
4. Find products above a certain price.
5. Sort products by price.
6. Find the top 5 expensive products.
7. Remove duplicate categories.

Then combine:

```
SELECT product_name, price
FROM products
WHERE price > 1000
ORDER BY price DESC
LIMIT 5;
```

The important thing is to teach him to **read SQL from top to bottom conceptually**, while understanding the logical processing order separately.

Session 2 — Aggregates

30 minutes

Teach:

COUNT
SUM
AVG
MIN
MAX

Start with:

```
SELECT COUNT(*)  
FROM orders;
```

Then:

```
SELECT  
    SUM(amount),  
    AVG(amount),  
    MIN(amount),  
    MAX(amount)  
FROM orders;
```

Then introduce:

GROUP BY
HAVING

Example:

```
SELECT  
    customer_id,  
    SUM(amount) AS total_spent  
FROM orders  
GROUP BY customer_id;
```

Then:

```
SELECT
  customer_id,
  SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 50000;
```

Key distinction to drill

WHERE → filters rows BEFORE aggregation

HAVING → filters groups AFTER aggregation

Give him 5–8 small questions around this distinction.

Session 3 — JOINS

30 minutes

This is probably the most important part of Day 1.

Teach the mental model first:

```
TABLE A
+
matching key
↓
TABLE B
```

Then:

INNER JOIN

```
SELECT
  c.customer_name,
  o.order_id,
  o.amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```

Then:

LEFT JOIN
RIGHT JOIN
FULL OUTER JOIN
SELF JOIN
CROSS JOIN

Don't spend equal time on all of them.

Prioritize:

INNER JOIN → LEFT JOIN → SELF JOIN → CROSS JOIN

Explain the others briefly if the database supports them.

Day 1 combination exercise

Give him a query like:

Find the top 5 customers by total order value.

He should gradually build:

```
SELECT
  c.customer_id,
  c.customer_name,
  SUM(o.amount) AS total_amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.customer_name
ORDER BY total_amount DESC
LIMIT 5;
```

This is important because it combines:

```
SELECT
+
JOIN
+
GROUP BY
+
```

SUM
+
ORDER BY
+
LIMIT

That is exactly the type of combination he needs to become comfortable with.

DAY 1 HOMEWORK

Keep it manageable: **45–60 minutes maximum.**

Give him **10 questions**:

Level 1

- 3 basic SELECT questions
- 2 WHERE questions

Level 2

- 2 aggregate questions
- 1 GROUP BY question

Level 3

- 2 JOIN + aggregate questions

Do **not** give 30 questions.

The objective is accuracy and confidence, not volume.

DAY 2 — SUBQUERIES → CTEs → CASE → ADVANCED JOINS

Goal

Move him from:

“I can write individual queries”

to:

“I can solve a multi-step SQL problem.”

Session 1 — Subqueries

Start with the simplest possible idea:

MAIN QUERY

↑

SUBQUERY provides a value

Example:

Find products whose price is greater than the average product price.

```
SELECT product_name, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
);
```

Then teach three forms:

Scalar subquery

```
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
)
```

IN subquery

```
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
)
```

Correlated subquery

Introduce only after the first two are comfortable.

Session 2 — CASE expressions

Teach:

```
CASE
  WHEN ...
  THEN ...
  WHEN ...
  THEN ...
  ELSE ...
END
```

Example:

```
SELECT
  product_name,
  price,
  CASE
    WHEN price >= 5000 THEN 'High'
    WHEN price >= 2000 THEN 'Medium'
    ELSE 'Low'
  END AS price_category
FROM products;
```

Then combine with aggregation:

```
SELECT
  CASE
    WHEN amount >= 5000 THEN 'High Value'
    ELSE 'Normal'
  END AS order_type,
  COUNT(*) AS order_count
FROM orders
GROUP BY
  CASE
    WHEN amount >= 5000 THEN 'High Value'
    ELSE 'Normal'
  END;
```

Session 3 — CTEs

Introduce CTEs as a **readability tool**, not another complicated feature.

```
WITH customer_sales AS (  
  SELECT  
    customer_id,  
    SUM(amount) AS total_sales  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT *  
FROM customer_sales  
WHERE total_sales > 50000;
```

Teach the mental model:

Step 1 → Build temporary result

↓

Step 2 → Query that result

Then compare:

Subquery

vs

CTE

The trainee doesn't need to memorize rules. He needs to understand:

“I can break a complicated problem into steps.”

Session 4 — Multi-table JOINS

Now combine:

customers

↓

orders

↓

order_items



products

Example:

```
SELECT
  c.customer_name,
  p.product_name,
  oi.quantity,
  oi.quantity * oi.unit_price AS sales
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id;
```

This is a major confidence-building exercise.

Day 2 challenge

Give him:

Find customers who have spent more than the average customer spending.

Let him struggle for 5–10 minutes.

Then guide him toward:

Step 1:

Calculate spending per customer



Step 2:

Calculate average customer spending



Step 3:

Compare each customer against that average

This teaches **problem decomposition**, which is much more valuable than memorizing syntax.

DAY 2 HOMEWORK

60–75 minutes

Give him approximately:

- 3 JOIN problems
- 2 GROUP BY problems
- 2 subquery problems
- 2 CASE problems
- 1 CTE problem

One problem should deliberately combine:

- JOIN
- +
- GROUP BY
- +
- HAVING
- +
- CASE

DAY 3 — WINDOW FUNCTIONS + COMPLETE SQL PROBLEM SOLVING

This is the critical day.

Goal

Make him capable of handling the more advanced SQL questions appearing in the regular class.

Session 1 — Window functions

Do **not** start by explaining the entire window-function theory.

Start with the problem:

“I want the total for each customer, but I still want to see every order.”

Aggregation:

```
SELECT
  customer_id,
  SUM(amount)
FROM orders
GROUP BY customer_id;
```

loses the individual order rows.

Window function:

```
SELECT
  order_id,
  customer_id,
  amount,
  SUM(amount) OVER (
    PARTITION BY customer_id
  ) AS customer_total
FROM orders;
```

This immediately explains **why window functions exist**.

Window functions to prioritize

Teach these first:

```
SUM() OVER()
AVG() OVER()
COUNT() OVER()
```

```
ROW_NUMBER()
RANK()
```

DENSE_RANK()

LAG()

LEAD()

Don't overwhelm him with every possible window function.

Session 2 — Ranking

This is particularly important.

Example:

```
SELECT
  employee_id,
  department_id,
  salary,
  RANK() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

Then explain:

PARTITION BY

↓

Which group?

ORDER BY

↓

How should that group be ordered?

RANK()

↓

What position does the row receive?

Then compare:

ROW_NUMBER

RANK

DENSE_RANK

Use a small example with duplicate salaries.

Session 3 — LAG / LEAD

Example:

```
SELECT
  order_date,
  amount,
  LAG(amount) OVER (
    ORDER BY order_date
  ) AS previous_amount
FROM orders;
```

Then:

amount - previous_amount

This gives him exposure to **time-series analysis**, which is extremely useful in data engineering and analytics.

Session 4 — The big combination

Now introduce the pattern he will need repeatedly:

```
JOIN
↓
FILTER
↓
GROUP
↓
WINDOW
↓
FILTER WINDOW RESULT
```

For example:

Find the top 3 customers in each region based on total sales.

Possible solution:

```
WITH customer_sales AS (  
  SELECT  
    c.region,  
    c.customer_id,  
    c.customer_name,  
    SUM(o.amount) AS total_sales  
  FROM customers c  
  JOIN orders o  
    ON c.customer_id = o.customer_id  
  GROUP BY  
    c.region,  
    c.customer_id,  
    c.customer_name  
)  
ranked_customers AS (  
  SELECT  
    *,  
    RANK() OVER (  
      PARTITION BY region  
      ORDER BY total_sales DESC  
    ) AS rnk  
  FROM customer_sales  
)  
SELECT *  
FROM ranked_customers  
WHERE rnk <= 3;
```

This single problem teaches:

```
JOIN  
+  
GROUP BY  
+  
SUM  
+  
CTE  
+  
WINDOW FUNCTION  
+  
PARTITION BY  
+  
RANK  
+
```

FILTER

That is exactly the type of **SQL synthesis** he needs.

The SQL Topics He Should Ultimately Cover

Your regular class may include topics beyond these, so I would use this as his catch-up checklist:

Area	Priority
SELECT / FROM	● Essential
WHERE	● Essential
DISTINCT	● Essential
ORDER BY	● Essential
LIMIT / TOP	● Essential
Aliases	● Essential
Arithmetic expressions	● Essential
NULL handling	● Essential
COALESCE / NULLIF	● High
CASE	● Essential
Aggregate functions	● Essential
GROUP BY	● Essential
HAVING	● Essential
INNER JOIN	● Essential

LEFT JOIN	 Essential
SELF JOIN	 High
CROSS JOIN	 Medium
UNION / UNION ALL	 High
Subqueries	 Essential
Correlated subqueries	 High
CTEs	 Essential
Recursive CTEs	 Later
Window functions	 Essential
ROW_NUMBER	 Essential
RANK / DENSE_RANK	 Essential
LAG / LEAD	 Essential
Date functions	 Essential
String functions	 High
Conditional aggregation	 Essential
EXISTS / NOT EXISTS	 High
INSERT	 High
UPDATE	 High
DELETE	 High
DDL	 Later
Views	 High
Temporary tables	 High

Transactions ● Later

Query optimization ● Later

Execution plans ● Later

For the **three-day rescue**, don't try to cover the entire table. Concentrate on the ● items.

Weekend Homework — "SQL Catch-Up Lab"

Instead of ordinary homework, give him **one mini-project**.

Business problem

You are a data analyst for an online retailer. Management wants a customer-sales report.

Give him these tables:

customers
orders
order_items
products

Ask him to produce:

Part A — Basic SQL

1. List all customers.
2. Find customers from a particular region.
3. Find orders above ₹10,000.
4. Find the 10 largest orders.

Part B — Aggregation

5. Calculate total sales.
6. Calculate sales by customer.
7. Calculate sales by product.

8. Find customers whose sales exceed ₹50,000.

Part C — JOINS

9. Display customer + order information.
10. Display customer + product + sales.
11. Find customers who have never placed an order.

Part D — Subqueries / CTE

12. Find customers spending more than the average customer.
13. Find products priced above the average product price.

Part E — Windows

14. Rank customers by sales.
15. Rank customers within each region.
16. Find the top 3 customers in every region.
17. Show each customer's previous order amount.

Final challenge

18. Produce:

Region
Customer
Total Sales
Regional Rank
Previous Order
Sales Category

This final query should require multiple SQL concepts.

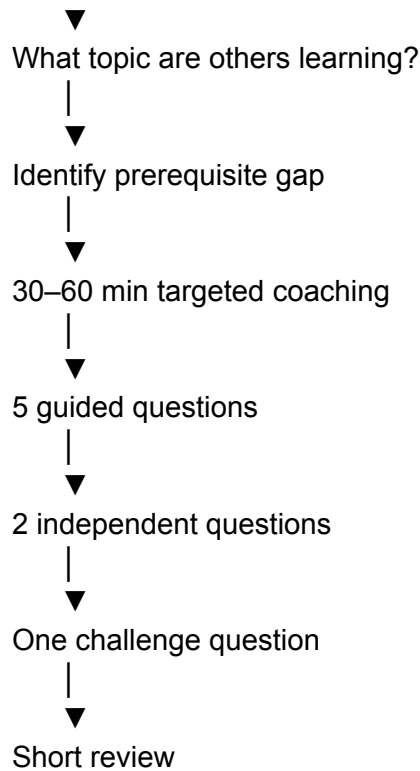
Most Important: Daily Teaching Pattern

I would **not** put him through another full classroom lecture.

Use a parallel track like this:

REGULAR CLASS

|



For example, if the regular class is doing window functions but he doesn't understand **GROUP BY**, don't teach window functions for two hours.

Spend 20 minutes fixing:

SELECT
→ GROUP BY
→ aggregate

then immediately bridge:

GROUP BY
vs
WINDOW FUNCTION

That makes the catch-up **parallel**, rather than creating a second curriculum.

A Crucial Confidence Strategy

Because he already feels that he isn't good enough, **avoid saying:**

"You are behind the others."

Instead say:

"You don't need to know everything. We are going to master the SQL patterns one by one."

And track progress visibly:

SQL RECOVERY SCORECARD

SELECT	✓
WHERE	✓
ORDER BY	✓
GROUP BY	✓
HAVING	●
INNER JOIN	✓
LEFT JOIN	●
SUBQUERY	●
CTE	●
CASE	✓
WINDOW FUNCTIONS	●
RANK	●
LAG/LEAD	●

Every time he masters something, change it to ✓.

The psychological effect is important: **he starts seeing evidence of progress rather than evidence of failure.**

The 3-Day Target

Don't expect him to become an SQL expert in three days.

Your realistic target is:

Before

"I don't know how to start this SQL problem."

After Day 3

"What does the problem require?"

Do I need:
JOIN?
GROUP BY?
SUBQUERY?
CTE?
WINDOW FUNCTION?

Let me break it into steps."

That is the real catch-up milestone.

Once he can decompose SQL problems, he can start keeping pace with the regular batch instead of depending on memorized queries.

And because migraines can make prolonged concentration difficult, I'd make the parallel track **flexible rather than forcing long sessions**; if symptoms are significant or worsening, the trainee should follow appropriate medical advice rather than trying to push through the training.